

BiteSplit

Group food ordering, without the math fight at the end.

A collaborative web app where a group orders from one shared cart in real time — and the bill splits itself, to the rupee, before anyone pays.

Turned the messiest part of group food ordering — the “who owes what” — into a self-resolving, real-time checkout.

Role UI/UX Designer & Frontend Developer (full-stack build with AI pair-programming)

Type Self-initiated proof of concept · 1-week sprint

Stack React 18 · TypeScript · Tailwind CSS · Framer Motion · Socket.io · Node / Express

The Problem

Ordering food for a group is a coordination tax everyone has paid and nobody enjoys. It breaks down in three predictable places:

- **One person becomes the human order form.** The host collects picks over chat, fixes the “wait, no onions,” and types it all into one account. Items get missed; the order goes in late.
- **The host eats the bill.** They pay upfront, then back-calculate what each person owes — items *plus* their share of delivery, tax, and tip — and chase people for ₹230 three days later.
- **Nobody can see the shared cart.** Delivery apps are single-device, so the group orders blind: duplicate fries, lopsided meals, no real-time alignment.

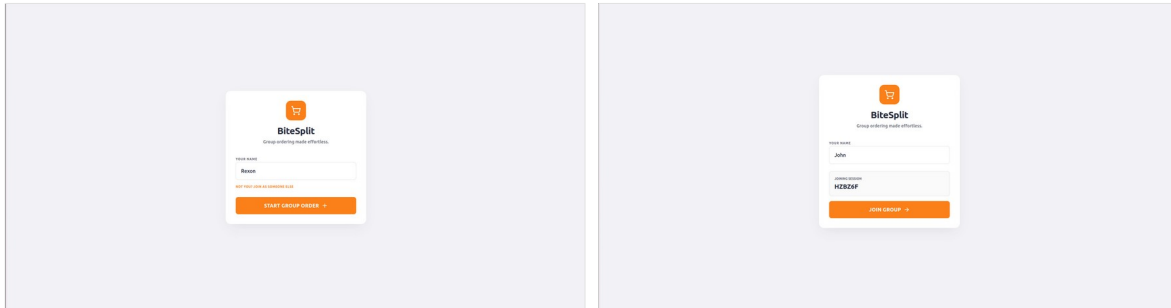
The Goal

Replace the chat + spreadsheet + IOU ritual with one shared session where anyone joins with **zero signup**, everyone sees the **same live cart**, and the bill **splits itself correctly** before checkout — so the host is never out of pocket and no one does mental math.

The Experience, Step by Step

1. Join in one field

The host starts a session and gets a short, shareable room code. Guests open the link, see the code pre-filled, and tap Join — no account, no password, no install. Removing friction at the door is why the group actually shows up.

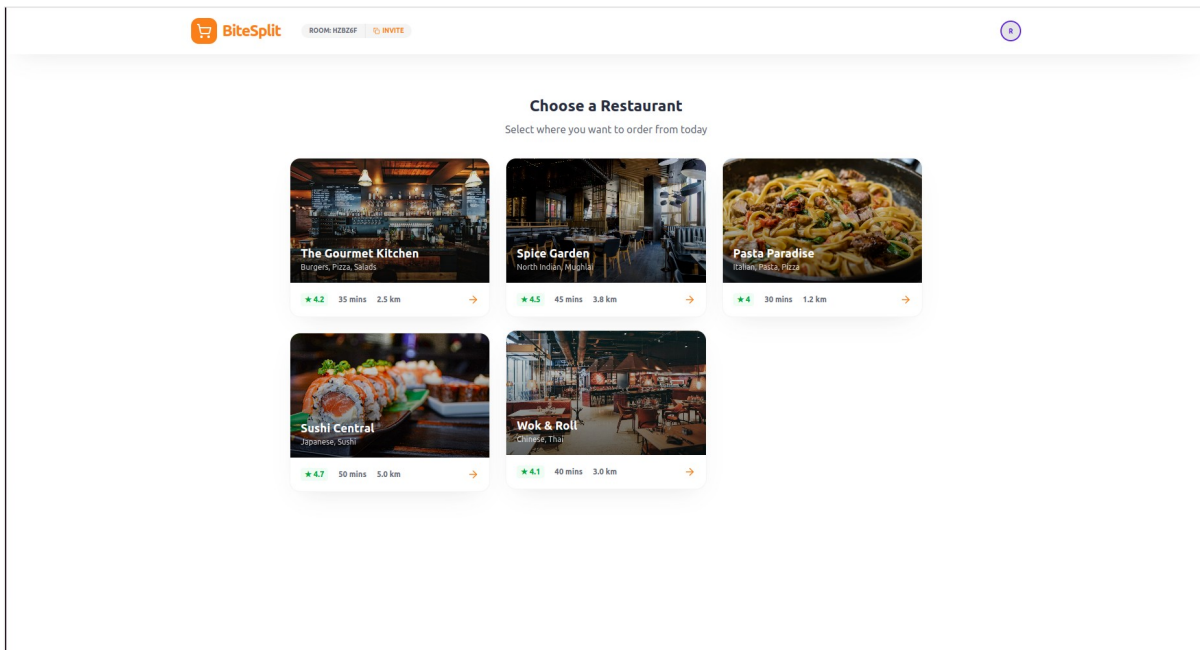


Host starts a group order with just a name.

Guests join the session with the code pre-filled.

2. Pick where you're ordering from

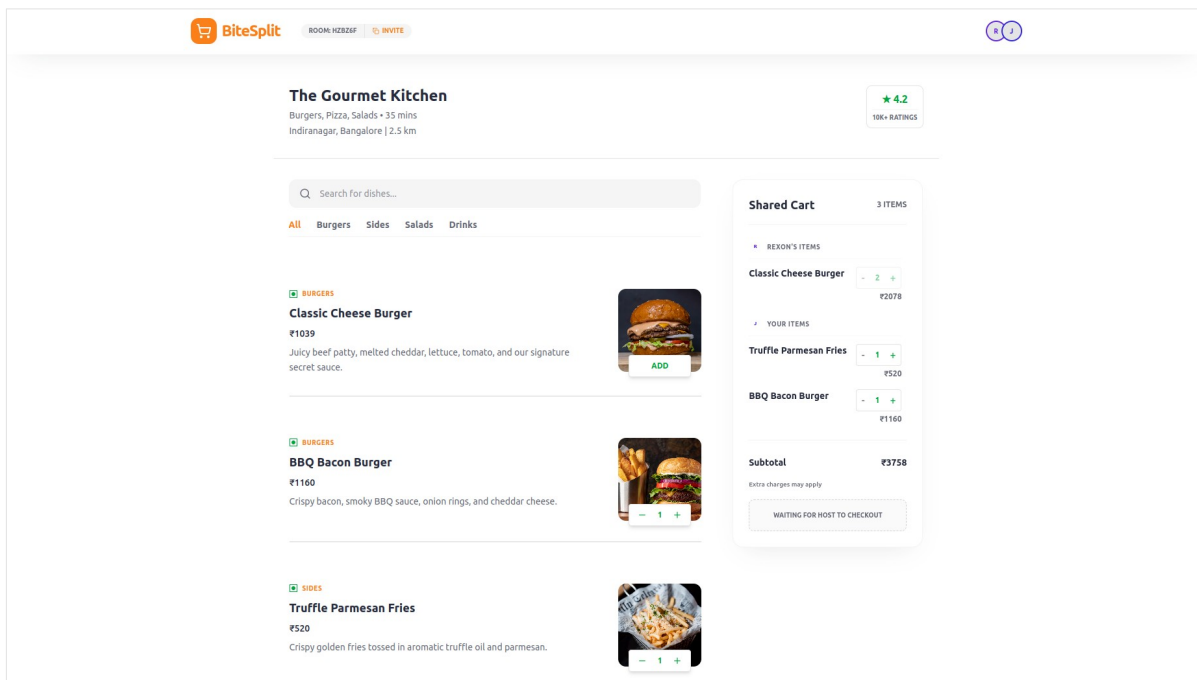
A familiar restaurant grid — rating, prep time, distance — so nobody needs instructions. Choosing one loads that restaurant's real menu, not a generic template.



Restaurant selection, shared across everyone in the room.

3. The shared cart is the centerpiece

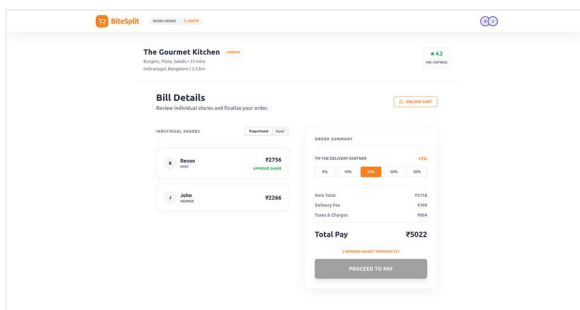
The menu sits on the left with search and category filters; the **Shared Cart** sits on the right and is the heart of the product. Items are grouped per person, so everyone sees exactly who ordered what — live. Every add, edit, and remove broadcasts instantly to all devices via Socket.io, which kills duplicate orders.



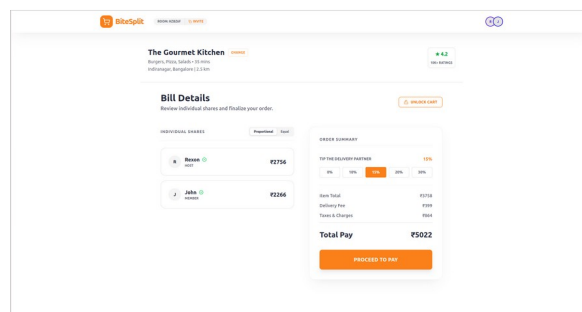
Live shared cart — items grouped by person, updating in real time for everyone.

4. The bill splits itself

At checkout the host gets **Individual Shares** with a Proportional / Equal toggle, a tip selector, and a transparent breakdown — items, delivery, taxes, total. Every member reviews and approves their own share, and the host *can't pay until everyone has* — the pay button stays disabled while “2 members haven't approved yet.”



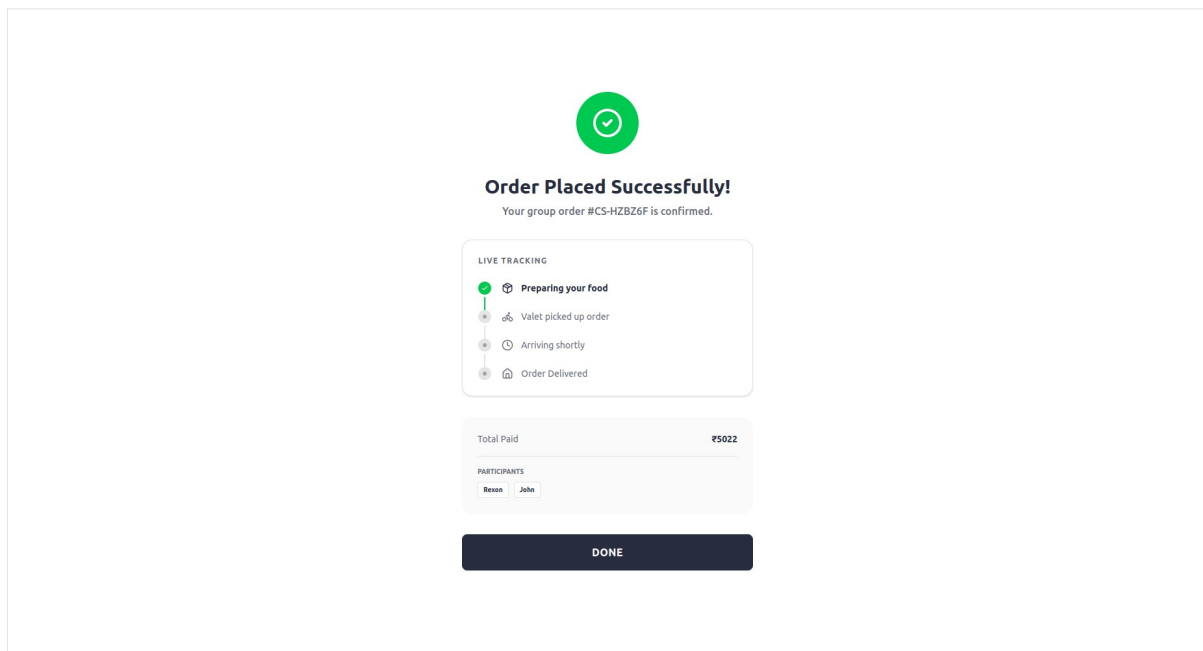
Host view — pay locked until all members approve.



All approved — Proceed to Pay unlocks.

5. Order placed, tracked together

Once paid, everyone lands on a shared confirmation with a live tracking timeline — Preparing → Picked up → Arriving → Delivered — plus the participant list and the total paid.



Shared order confirmation and live tracking for the whole group.

Key Design Decisions

- **Make the cart the hero, not the menu.** In a *group* app the shared cart is the whole reason you're here, so it gets persistent real estate — a sticky sidebar on desktop, a slide-up sheet on mobile.
- **Borrow a visual language people trust.** Money and other people are involved; novelty would cost trust. Clean white surfaces, a confident orange accent, and clear typography buy instant comfort.
- **Built for two contexts at once.** Guests join on phones; the host often checks out on a laptop — so the layout is genuinely fluid, not one design squeezed into the other.
- **The host is a referee, not a gatekeeper.** The host controls flow (lock, checkout, pay) but never people's choices — members approve their own shares.
- **No minimum order.** A ₹520 side of fries is a valid order; the product serves the lightest member as gracefully as the biggest.

The Hard Part: Getting the Money Exactly Right

A bill splitter that's off by a rupee isn't a rounding quirk — it's a broken promise, because the host absorbs the gap.

- **Floating-point money is a trap.** $0.1 + 0.2$ is 0.30000000000000004 . So calculations run on integers in the smallest unit (paise) and only convert back to rupees for display.
- **The split has to reconcile.** Proportional or equal, the sum of all shares equals $\text{subtotal} + \text{delivery} + \text{taxes} + \text{tip}$ exactly — the stray rounding rupee is

assigned deterministically.

- **The guarantee.** A ₹3,758 cart became a ₹5,022 total split as ₹2,756 + ₹2,266 — and those add back to ₹5,022 exactly. That reconciliation is the feature.

Edge Cases I Designed For

- **Host switches restaurant mid-session** → all carts and totals clear instantly, so menus can't cross-contaminate.
- **A guest refreshes or drops connection** → room state is preserved and they rejoin straight into their own cart.
- **Cart locked at checkout** → adds and removes freeze, so no one alters the order after shares are computed.
- **Someone leaves the group** → every remaining member's split is recomputed live.

Outcome & Takeaways

- **~15 minutes → seconds.** Group coordination collapses from a long chat thread to everyone tapping on their own phone.
- **Host anxiety, designed out.** Accurate shares, visible approvals, and a guarantee they're never short.
- **Full stack, solo, in a week.** Real-time sync, a self-reconciling split engine, and a complete join-to-delivery flow — proof that AI pair-programming lets one designer own the whole product while still deciding what's correct and what feels right.

Designed & built by Visvesvaran K — UI/UX Designer & Frontend Developer.